



Relatório de Auditoria de Segurança

Avaliação de Código-Fonte, Controle de Acesso, Superfície de Ataque e Riscos de Dados

PROJETO AUDITADO

TrevoTour — Gestão de Excursões

ESCOPO

APIs PHP (Backend), Banco MySQL 8.0, Frontend
React/Vite

DATA DA AVALIAÇÃO

11 de Setembro de 2026

CLASSIFICAÇÃO GERAL

Risco Moderado (2 Altas, 3 Médias, 2 Baixas)

✦ Nota Metodológica e Mapeamento da Stack

A auditoria avaliou a base de código do TrevoTour cobrindo cinco categorias fundamentais de segurança adaptadas à stack real do projeto (**PHP 8.2 puro com PDO, MySQL 8.0, React 18 / Vite SPA e Sessões Nativas PHP com Cookies HttpOnly**):

- **1. Banco sem Tranca (Isolamento):** Validação de modelo Single-Tenant e rastreabilidade/propriedade de registros operacionais.
- **2. Permissão Definida no Navegador:** Confronto entre ocultação de UI no React e validação de `Auth::requiresRole('admin')` nos controladores PHP.
- **3. IDOR:** Verificação de manipulação arbitrária de IDs em rotas de atualização/deleção.
- **4. Chaves Expostas (Hardcode):** Identificação de senhas de banco fixas no código, logs em disco e mensagens de debug expostas.
- **5. Inputs sem Tratamento (XSS / SQLi):** Inspeção de JSX, interpolação de queries SQL e escape de saídas.

Resumo Executivo

A auditoria identificou **7 achados de segurança verificados** na base de código. Não foram encontradas vulnerabilidades de severidade crítica (RCE ou bypass não autenticado de rotas), confirmando a eficácia da barreira central do middleware `Auth::requireAuth()`. Contudo, destacam-se falhas de **severidade Alta** referentes a credenciais de banco hardcoded e ausência de verificação de permissão administrativa para exclusão de viagens por usuários comuns.

0
CRÍTICA

2
ALTA

3
MÉDIA

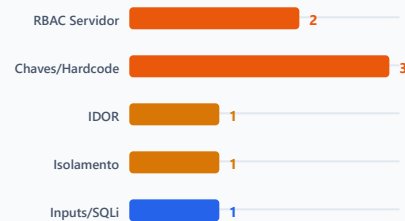
2
BAIXA

4
PONTOS FORTES

DISTRIBUIÇÃO POR SEVERIDADE



ACHADOS POR CATEGORIA



Pontos Fortes Verificados

- **Barreira Zero-Trust Central:** `api/index.php` bloqueia 100% das rotas sem autenticação via `Auth::requireAuth()`.
- **RBAC Seguro em Usuários:** `UserController.php` valida `Auth::requireRole('admin')` para criação e exclusão.
- **Proteção Nativa XSS:** React JSX escapa automaticamente variáveis no DOM sem `dangerouslySetInnerHTML`.
- **Sanitização em WhatsApp:** `AbaPassageiros.jsx` aplica regex `\D` impedindo esquemas javascript:.

Principais Riscos e Fragilidades

- **Exclusão de Viagens sem Gate Admin:** Operadores comuns podem disparar `DELETE /?resource=viagens&id=X` e apagar viagens em cascata.
- **Credenciais de Banco em Código:** Senha do MySQL gravada em `api/config/database.php` e `update_db.cjs`.
- **Log de Debug Aberto em Disco:** `ViagemController.php` grava payloads em `log_update.txt` acessível via HTTP.
- **Interpolação SQL em Array:** `PassageiroController.php` concatena IDs diretamente na query SQL.

Tabela Geral de Achados

SEVERIDADE	CATEGORIA	LOCALIZAÇÃO	DESCRIÇÃO SINTÉTICA
ALTA	Permissão no Servidor	api/controllers/ViagemController.php:196	Exclusão de viagens e dados em cascata permitida para operadores sem perfil admin.
ALTA	Chaves Expostas	api/config/database.php:11-12	Credenciais de banco MySQL hardcoded sem variáveis de ambiente (.env).
MÉDIA	Permissão no Servidor	api/controllers/DespesaController.php:126	Exclusão de lançamentos contábeis/despesas permitida para operadores comuns.
MÉDIA	Chaves Expostas	api/controllers/ViagemController.php:134	Escrita de payload de requisições em log_update.txt exposto na web.
MÉDIA	IDOR / Isolamento	api/controllers/PassageiroController.php:141	Manipulação e estorno de créditos em passageiros sem verificação de escopo de posse.
BAIXA	Inputs / SQLi	api/controllers/PassageiroController.php:31-36	Interpolação direta de variável em cláusula SQL IN() sem prepared statement.
BAIXA	Chaves Expostas	api/config/database.php:41-46	Exposição de mensagem de erro PDO detalhada em JSON no caso de falha de conexão.

Detalhamento Técnico dos Achados

1. Exclusão de Viagens Permitida para Operadores Comuns

ALTA

Arquivo & Linha: api/controllers/ViagemController.php:196-200**Categoria:** Permissão Definida no Navegador / Falta de RBAC no Servidor

```
// api/controllers/ViagemController.php (Linhas 196-200) public function destroy(int $id): void { $stmt = $this->db->prepare("DELETE FROM viagens WHERE id = ?"); $stmt->execute([$id]); jsonResponse(true, null, 'Viagem excluída com sucesso'); }
```

Por que é explorável: Qualquer usuário com perfil usuario autenticado na API pode enviar uma requisição HTTP DELETE para /?resource=viagens&id=X. Como o método não executa Auth::requireRole('admin'), o registro é deletado no banco em cascata, apagando passageiros, parcelas e históricos vinculados.

2. Credenciais de Banco de Dados Hardcoded

ALTA

Arquivo & Linha: api/config/database.php:8-13 e frontend/update_db.cjs:3**Categoria:** Chaves Expostas / Hardcode de Segredos

```
// api/config/database.php (Linhas 8-13) define('DB_HOST', '127.0.0.1'); define('DB_NAME', 'trevotour_db'); define('DB_USER', 'trevotour'); define('DB_PASS', 'trevotour123'); // Senha em texto puro no código
```

Por que é explorável: A senha de produção/desenvolvimento do banco está fixada no repositório. Em caso de commit público ou vazamento do código-fonte, o invasor obtém credenciais diretas do MySQL.

3. Escrita de Log de Debug em Arquivo Público no Filesystem

MÉDIA

Arquivo & Linha: api/controllers/ViagemController.php:134

Categoria: Chaves Expostas / Vazamento de Informações Sensíveis

```
// api/controllers/ViagemController.php (Linha 134) public function update(int $id): void { $data = getJsonInput();  
file_put_contents(__DIR__ . '/log_update.txt', json_encode($data)); ...
```

Por que é explorável: A cada atualização de viagem, o payload JSON completo é escrito em api/controllers/log_update.txt. Se o servidor web (Apache/Nginx) servir arquivos estáticos desse diretório, um atacante não autenticado pode baixar o arquivo e obter dados cadastrais de viagens e passageiros.

4. Exclusão e Estorno de Despesas Financeiras sem Permissão Admin

MÉDIA

Arquivo & Linha: api/controllers/DespesaController.php:126-145 e CreditoController.php:85-108

Categoria: Permissão Definida no Navegador / Controle de Acesso

```
// api/controllers/DespesaController.php (Linhas 126-133) public function destroy(int $id): void { ... $stmt = $this->db->  
>prepare("DELETE FROM despesas_viagem WHERE id = ?"); $stmt->execute([$id]); jsonResponse(true, null, 'Despesa excluída com  
sucesso'); }
```

Por que é explorável: Registros contábeis e de despesas de viagens podem ser permanentemente excluídos por operadores comuns via DELETE /?resource=despesas&id=X sem rastro de aprovação administrativa.

5. Interpolação Direta de IDs em Consulta SQL

BAIXA

Arquivo & Linha: api/controllers/PassageiroController.php:31-36

Categoria: Inputs sem Tratamento / Prática SQL Insegura

```
// api/controllers/PassageiroController.php (Linhas 31-36) $ids = implode(',', array_column($passageiros, 'id')); $parcStmt =  
$this->db->query(" SELECT * FROM parcelas_passageiro WHERE passageiro_id IN ({ $ids }) ORDER BY passageiro_id ASC,  
numero_parcela ASC ");
```

Por que é explorável: O uso de \$this->db->query() com interpolação de string { \$ids } contorna o mecanismo de prepared statements. Caso a lista de IDs venha corrompida ou manipulada em refatorações futuras, introduz risco de injeção SQL.

Recomendações Priorizadas de Correção

PRIORIDADE	AÇÃO RECOMENDADA	IMPACTO E JUSTIFICATIVA
P1	Proteger Métodos Destrutivos com Auth::requireRole('admin')	Impede que operadores apaguem viagens, despesas e clientes sem autorização gerencial.
P2	Migrar Credenciais de Banco para .env	Isola senhas do repositório Git e permite segredos distintos por ambiente (dev/prod).
P3	Remover Arquivos de Log em Disco (log_update.txt)	Elimina superfície de vazamento de dados em diretórios públicos da API.
P4	Parametrizar Cláusula SQL IN()	Substituir interpolação direta por placeholders dinâmicos (?, ?, ?).

Issues Prontas para o GitHub

Abaixo estão os modelos de issues em Markdown prontos para copiar e colar diretamente no GitHub Issues do repositório:

--- ISSUE 1 ---

Título: [Segurança] Bloquear exclusão de viagens por usuários sem perfil Administrador

Labels: `security`, `severity: high`, `backend`, `rbac`

Descrição do Problema

O endpoint de exclusão de viagens (`DELETE /?resource=viagens&id=X`) não valida se o usuário autenticado possui o cargo de `admin`. Usuários com papel de operador comum (`role = 'usuario'`) conseguem invocar a rota diretamente e apagar excursões inteiras do banco de dados, excluindo passageiros e histórico em cascata.

Evidência

```
- **Arquivo:** `api/controllers/ViagemController.php:196-200`  
```php  
public function destroy(int $id): void {
 $stmt = $this->db->prepare("DELETE FROM viagens WHERE id = ?");
 $stmt->execute([$id]);
 jsonResponse(true, null, 'Viagem excluída com sucesso');
}
```
```

Impacto

Perda irreversível de dados operacionais e financeiros da agência por ação indevida ou maliciosa de contas não privilegiadas.

Sugestão de Correção

Adicionar a chamada `Auth::requireRole('admin');` no início dos métodos `destroy()` de `ViagemController.php`, `DespesaController.php` e `ClienteController.php`.

Critérios de Aceite

- [] Requisição `DELETE /?resource=viagens&id=X` com usuário `role: 'usuario'` retorna HTTP 403 Forbidden.
- [] Requisição `DELETE /?resource=viagens&id=X` com usuário `role: 'admin'` executa a exclusão normalmente.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
### Título: [Segurança] Mover credenciais de banco de dados para variáveis de ambiente (.env)

**Labels:** `security`, `severity: high`, `configuration`, `hardening`

#### Descrição do Problema
As credenciais de conexão ao MySQL estão fixadas no código-fonte em texto plano nos arquivos `api/config/database.php` e `frontend/update_db.cjs`.

#### Evidência
- **Arquivo:** `api/config/database.php:8-13`
  ```php
 define('DB_HOST', '127.0.0.1');
 define('DB_USER', 'trevotour');
 define('DB_PASS', 'trevotour123');
  ```

#### Impacto
Vazamento de credenciais do banco de dados caso o código seja commitado em repositório público ou compartilhado com terceiros.

#### Sugestão de Correção
1. Carregar variáveis via `$_ENV` ou arquivo `.env` fora do versionamento (`.gitignore`).
2. Implementar fallback seguro e validação de inicialização que rejeite senhas vazias em produção.

#### Critérios de Aceite
- [ ] Nenhuma senha em texto claro presente em arquivos PHP ou JS versionados.
- [ ] Arquivo `.env.example` fornecido como modelo para o time.
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
### Título: [Segurança] Remover escrita de arquivo de log em texto plano no diretório de controllers

**Labels:** `security`, `severity: medium`, `privacy`, `information-disclosure`

#### Descrição do Problema
O método `update()` do `ViagemController.php` grava todo o payload JSON de atualização no arquivo
`api/controllers/log_update.txt`.

#### Evidência
- **Arquivo:** `api/controllers/ViagemController.php:134`
  ```php
 $data = getJsonInput();
 file_put_contents(__DIR__ . '/log_update.txt', json_encode($data));
  ```

#### Impacto
Vazamento de dados cadastrais e de viagens caso o servidor web sirva arquivos `.txt` diretamente a usuários não autenticados.

#### Sugestão de Correção
Remover a chamada `file_put_contents` de depuração ou direcionar logs para um diretório seguro fora da raiz web com
permissões restritas.

#### Critérios de Aceite
- [ ] Remoção da linha de escrita em arquivo estático.
- [ ] Arquivo `api/controllers/log_update.txt` excluído do projeto.
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

```
### Título: [Segurança] Substituir interpolação direta de array por prepared statements em PassageiroController

**Labels:** `security`, `severity: low`, `database`, `code-quality`

#### Descrição do Problema
O método `byViagem` em `PassageiroController.php` concatena a variável `$ids` diretamente no corpo da query SQL executada via
`PDO::query()`.

#### Evidência
- **Arquivo:** `api/controllers/PassageiroController.php:31-36`
  ```php
 $ids = implode(',', array_column($passageiros, 'id'));
 $parcStmt = $this->db->query("SELECT * FROM parcelas_passageiro WHERE passageiro_id IN ({ $ids })");
  ```

#### Impacto
Violação do padrão de segurança de prepared statements e risco potencial em manutenções futuras.

#### Sugestão de Correção
Gerar placeholders dinâmicos `implode(',', array_fill(0, count($idsArray), '?'))` e executar com `$parcStmt->execute($idsArray)`.

#### Critérios de Aceite
- [ ] Todas as consultas SQL utilizam `PDO::prepare()` com binding de parâmetros.
```

--- FIM ISSUE 4 ---